# Graph Topology Augmentation for Prioritized Sweeping in RL Planning

Research Memo – Gia-Hung Pham – April 2026

## Setting

**Environment Setting** MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, \mathcal{R}, \gamma)$ with transition kernel $p$ *known* and reward $r_t : \mathcal{S} \to \mathbb{R}$ *state-based and nonstationary*. Because $p$ is known, the graph topology is fully available offline. Because $r_t$ is nonstationary, the agent must replan repeatedly – standard VI sweeps are too expensive per replanning cycle. This setting is met in various problems in logistics scheduling and robotic navigation. The environment can be *stochastic* (non-deterministic transitions and rewards) and/or *nonstationary* (changing transition and reward structure) obeying some underlying dynamics. We focus on the nonstationary reward setting, where we know (or learn) a static transition structure, without encouraging prior learning of the environment's underlying generating dynamics.

**Agent Setting** An *agent* is a decision-making system that navigates the MDP, observes states, chooses actions, and updates its policy to maximize long-run discounted reward. It may know the exact MDP structure, or learn it from experience, or not use it at all. A *model-based planning agent* additionally uses a known or learned transition model to simulate outcomes and allocate value-function backups before acting. For that, all the agent can perceive is the static current reward function and the transition structure; plus the history of observed reward shifts, which may or may not be informative about future shifts.

**Value Prediction and Prioritized Sweeping** The agent has an (explicit or implicit) rule of performing action as to navigate the environment, which is called the *planning policy* $\pi_t$. The agent can maintains a value function $V_t$ that estimates the expected return from each state under the current policy $\pi_t$.

When the reward function or the policy changes, $V_t$ becomes inaccurate and must be updated to reflect the new reward landscape. The standard approach is to perform full value iteration (VI) sweeps over the entire state space, but this is computationally expensive because of the inefficient allocation of updates. Reward (change) is usually sparse and local, so performing updates uniformly over the state space hinders the propagation of reward information to the states that matter most for decision-making.

*Prioritized sweeping* is the planning heuristic that orders these backups by estimated impact, typically using reward residuals and predecessor propagation. The agent's *replanning budget* is the number of prioritized backups it can perform between each action selection/execution step. The agent must prioritize carefully to optimize the impact of the learning process (convergence, adaptation, etc.) within this budget.

**Problem Statement** Given the above setting, we want to design a *prioritized sweeping module* that can be integrated into any model-based planning agent to accelerate replanning under nonstationary reward. Concretely, the module should augment prioritized sweeping with graph-topological information extracted from the known transition structure, so that state updates are ranked not only by reactive Bellman error but also by anticipated topology-induced reward propagation. The goal is to spend a limited replanning budget on the states most likely to matter after each reward shift.

There are two main RL algorithms that uses tabular prioritized sweeping: Dyna-Q and Value Iteration. We focus on the latter, specially for theoretical analysis, as it is more directly amenable to theoretical analysis. However, the integration for both is sensibly the same, the core principles of GTA-PS should also apply seamlessly.

We propose the framework of **Graph Topology Augmentation (GTA)**, leveraging tools from Spectral Graph Theory. Our contribution centers on the particular algorithm of GTA-PS – one of the many realizations we can design – focusing on ordering state updates in prioritized sweeping. This algorithm replaces reactive residual priority with a proactive topology-aware priority.

**Assumptions**

1. MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, \mathcal{R}, \gamma)$ with finite state space $\mathcal{S}$ and action space $\mathcal{A}$.
2. $r_t : \mathcal{S} \to \mathbb{R}$, $r_t(s) \sim \mathcal{R}(s,t)$, is *state-based* (goal-based), **known** and **bounded** (so that the advantage function in Assm. 8 is bounded). This is a common setting in nonstationary RL, where the reward function changes due to external factors (e.g., changing customer demand in logistics, shifting task priorities in robotics) rather than being learned from data.
3. The reward distribution $r_t \sim \mathcal{R}(t)$ is **nonstationary** and **unknown** – as we do not encourage prior learning of the reward shift pattern, the agent must adapt online to arbitrary reward shifts.
4. The reward vector $\mathbf{r}$ is **known**, shifts at an average rate of $\frac{1}{T}$ (a constraint mainly for concreteness) with maximum discrepancy $|\delta\mathbf{r}|_\infty \le \Delta_r$ and indicate to be *sparse* (hard for reactive PS).
5. $p = p(s'|s,a)$ or $\hat{p} = \hat{p}(s'|s,a)$ is known, tractable, or learnable; $V_t/Q_t$ (host (action-)value function), $\pi_t$ (host policy) evolve during training. In the known-$p$ case (the focus of this paper) all spectral quantities are exact; The "learnable" branch substitutes $\hat{p}$ for $p$, is used for empirical testing only.
6. The GTA-PS module maintains a dedicated *priority policy* $\pi_t^{\mathrm{PS}}$, separate from the host algorithm's planning policy $\pi_t$.
7. The chain induced by $\pi_t^{\mathrm{PS}}$, $P_t^{\mathrm{PS}}(s,s') = \sum_a \pi_t^{\mathrm{PS}}(a|s)\, p(s'|s,a)$, is ergodic for all $t$; ergodicity of $\pi_t$ is not required.
8. *(GTA-PS topology policy.)* $\pi_t^{\mathrm{PS}}$ is a Boltzmann policy over current action advantages with fixed module temperature $b > 0$:

$$\pi_t^{\mathrm{PS}}(a|s) = \frac{\exp(b\, A_t(s,a))}{\sum_{a'} \exp(b\, A_t(s,a'))}, \qquad A_t(s,a) = \gamma \textstyle\sum_{s'} p(s'|s,a)\, V_t(s') - \max_{a'} \gamma \sum_{s'} p(s'|s,a')\, V_t(s').$$

   Here $A_t(s,a) \le 0$ and the best action at state $s$ has zero logit, providing a numerically stable state-centered softmax. $\pi_t^{\mathrm{PS}}$ is used solely to construct the topology-induced priority operator $\mathbf{f}_t^{out}$, $\mathbf{f}_t^{in}$. After each Bellman backup at state $s$ that changes $V(s)$ by $\Delta V$, the GTA-PS module recomputes $\pi^{\mathrm{PS}}(\cdot|s)$ from the updated advantages $A_{t+1}(s,\cdot)$.
9. The module temperature satisfies $b \le b_{\max} < \infty$ (finite, excluding zero temperature), ensuring $\pi_t^{\mathrm{PS}}(a|s) \ge \pi_{\mathrm{act,min}} > 0$ for all $(s,a,t)$. Under the additional conditions that the MDP is *communicating* and *aperiodic*, this guarantees ergodicity of $P_t^{\mathrm{PS}}$ (Assumption 5) and enables the SLEM perturbation bounds (Section 3). Communicating alone guarantees irreducibility and a unique stationary distribution, but not aperiodicity; a periodic communicating chain has $\mathrm{SLEM}(P_t) = 1$ and is not ergodic.

*Note.* By ergodicity of $P_t^{\mathrm{PS}}$, there exists a unique stationary *state* distribution $\mu_t^{\mathrm{PS}} : \mathcal{S} \to \mathbb{R}_{>0}^{|\mathcal{S}|}$ This $\mu_t^{\mathrm{PS}}$ is a distribution over *states*. Also the probability is bounded, as we define $\mu_{\min} \triangleq \min_s \mu_t^{\mathrm{PS}}(s) > 0$ and $\mu_{\max} \triangleq \max_s \mu_t^{\mathrm{PS}}(s) \le 1$.

**Clarifications for Key Notation and Concepts**

**Scope of $\mathbf{r}_t$ and $\delta\mathbf{r}_t$.** For notational uniformity, $\mathbf{r}_t$ always denotes the reward vector currently held by the host agent's knowledge of the current step. $\delta\mathbf{r}_t$ denotes the update that changes that vector. The linear identities

$$\mathbf{f}_t^{out} = (L_t^{out} + \varepsilon I)^{-1}\mathbf{r}_t,$$
$$\mathbf{f}_t^{in} = (L_t^{in} + \varepsilon I)^{-1}\mathbf{r}_t$$

and the corresponding update formulas with $\delta\mathbf{r}_t$ therefore apply in both the exact-planning and Dyna deployments. What changes between the two settings is not the operator, but how the host agent acquires the current reward vector.

**Exact-planning / DP interpretation.** Unless stated otherwise, the theoretical sections use the known-$p$ planning setting. In that regime, $\mathbf{r}_t$ is the exact state-reward vector from the current MDP specification, available to the planner at time $t$. A reward change is an exogenous environment shift, so

$$\delta\mathbf{r}_t = \mathbf{r}_{t+1} - \mathbf{r}_t$$

is known exactly when the shift occurs. Hence the topology updates induced by reward change are exact:

$$\delta\mathbf{f}_t^{out} = (L_t^{out} + \varepsilon I)^{-1}\delta\mathbf{r}_t,$$
$$\delta\mathbf{f}_t^{in} = (L_t^{in} + \varepsilon I)^{-1}\delta\mathbf{r}_t.$$

**Dyna / learned-model interpretation.** In the Dyna deployment the agent does not start with an oracle reward vector. Instead, reward information is obtained through sampled or learned model queries together with next-state information, and the topology layer maintains a running state-based aggregate $\mathbf{r}_t$ consistent with the state-based reward assumption of this memo. Whenever a new query revises the current reward estimate at state $s$, the induced discrepancy is treated as

$$\delta\mathbf{r}_t = \big(\widehat{r}_t^{\,\mathrm{new}}(s) - \widehat{r}_t^{\,\mathrm{old}}(s)\big)e_s,$$

or more generally as a sparse correction over the affected support. This update may come from either a genuine nonstationary shift in the environment or a newly discovered / corrected reward estimate at a previously uncertain state. The topology layer handles both cases identically: it updates the current aggregate $\mathbf{r}_t$ and then applies the same linear resolvent update to obtain $\delta\mathbf{f}_t^{out}$ and $\delta\mathbf{f}_t^{in}$.

**Implementation note.** There is therefore no contradiction between the DP and Dyna uses of the notation. In DP, $\mathbf{r}_t$ is exact and $\delta\mathbf{r}_t$ is an announced exogenous shift. In Dyna, $\mathbf{r}_t$ is the current reward estimate assembled from model experience, and $\delta\mathbf{r}_t$ is any discrepancy that changes that estimate, regardless of whether it came from exploration or nonstationarity. The detailed interactions are described in the pseudocodes for each integration in the later sections.

## Directed Spectral Theory of the Policy-Induced Chain

**Policy-induced kernel.** Let $P^a(s,s') = p(s'|s,a)$ be the known per-action matrices. The GTA-PS topology policy $\pi_t^{\mathrm{PS}}$ induces the row-stochastic transition kernel

$$P_t^{\mathrm{PS}} = \sum_a \mathrm{diag}(\pi_t^{\mathrm{PS}}(\cdot,a)) \cdot P^a, \qquad P_t^{\mathrm{PS}}\mathbf{1} = \mathbf{1}.$$

For brevity we write $P_t \equiv P_t^{\mathrm{PS}}$ throughout Sections 3–8 when the context is unambiguous. $P_t$ is generically non-reversible: the topology policy concentrates mass on preferred directions, creating asymmetric flow between states. Imposing symmetry discards this directional information. We work directly with the directed structure.

**The directed Laplacian pair.** Two natural Laplacians are defined on the same directed graph:

$$L_t^{out} = I - P_t, \qquad L_t^{in} = D_t^{in} - P_t^\top, \qquad (D_t^{in})_{ss} = \textstyle\sum_{s'}(P_t)_{s's}.$$

$L_t^{out}$ is the *out-Laplacian*: rows encode out-flow divergence. $L_t^{in}$ is the *in-Laplacian*: it equals the out-Laplacian of the time-reversed graph, so rows encode in-flow convergence. Their null structures differ:

| Laplacian | Right null | Left null |
|---|---|---|
| $L_t^{out} = I - P_t$ | $\mathbf{1}$ | $(\mu_t^{\mathrm{PS}})^\top$ |
| $L_t^{in} = D_t^{in} - P_t^\top$ | $\mathbf{1}$ | non-trivial* |

**Discrete Green's Function and Topological Potentials.** The directional spread of reward potentials across the graph is formally governed by the directed graph Laplacian and algebraically captured by its $\varepsilon$-regularised inverse, the discrete Green's function (Chung and Yau, 2000). This formulation solves the associated boundary-value conditions on finite graphs by generating the exact fundamental matrix for the equivalent discounted random walk. Applying the Green's function maps any local transition or reward perturbation into a global, topology-aware potential field across the entire finite chain structure.

**SLEM as the topology statistics.** For a non-reversible $P_t$, eigenvalues of $L_t^{out}$ may be complex, making the Fiedler value ill-defined as a scalar trigger. The correct directed analogue is the *Second Largest Eigenvalue Modulus*:

$$\mathrm{SLEM}(P_t) = \max\big\{|\lambda| : \lambda \in \sigma(P_t),\ \lambda \neq 1\big\} \in [0,1).$$

When the underlying operator is clear, we may write simply "SLEM" in prose, but it always means $\mathrm{SLEM}(P_t)$, the second-largest eigenvalue modulus of the policy-induced transition kernel $P_t$. SLEM is real and well-defined for any ergodic chain, but for general *non-reversible* chains a rigorous mixing-time upper bound is better expressed through *multiplicative reversiblization*.

Let $P_t^*$ denote the adjoint of $P_t$ in $L_2(\mu_t^{\mathrm{PS}})$, and define the multiplicative-reversiblization gap

$$\mathrm{gap}_{\mathrm{mr}}(P_t) \triangleq 1 - \lambda_2(P_t^* P_t) = 1 - \sigma_2(P_t)^2,$$

where $\sigma_2(P_t)$ is the second singular value of $P_t$ in $L_2(\mu_t^{\mathrm{PS}})$. Then the policy-induced chain obeys the rigorous non-reversible bounds

$$\frac{1-2\varepsilon}{2\Phi(P_t)} \ \le\ t_{\mathrm{mix}}(\varepsilon)\ \le\ \frac{2}{\mathrm{gap}_{\mathrm{mr}}(P_t)}\log\Big(\frac{1}{2\varepsilon\sqrt{\mu_{\min}}}\Big)$$

where $\Phi(P_t)$ is the conductance of the policy-induced chain and $\mu_{\min} \triangleq \min_s \mu_t^{\mathrm{PS}}(s)$. The upper bound uses the $L_2(\mu_t^{\mathrm{PS}})$ contraction of $P_t$ through $P_t^* P_t$; the lower bound is the standard bottleneck bound.

For reversible lazy $P_t$: $\mathrm{SLEM}(P_t) = 1 - \lambda_2(L_t^{\mathrm{sym}})$ exactly, so $\mathrm{SLEM}(P_t)$ strictly generalizes the Fiedler value. In the non-reversible setting $\mathrm{SLEM}(P_t)$ is best viewed as a cheap online geometric proxy: small SLEM indicates rapid contraction of non-trivial modes, while larger SLEM indicates a persistent slow mode, often caused by bottlenecks or near-periodic recirculation, even though the rigorous upper bound is governed by $\mathrm{gap}_{\mathrm{mr}}(P_t)$ rather than SLEM alone.

**Biorthogonal spectral decomposition.** For diagonalizable $P_t$, write the biorthogonal eigendecomposition:

$$P_t v_k = \lambda_k v_k, \quad u_k^\top P_t = \lambda_k u_k^\top, \quad u_j^\top v_k = \delta_{jk}, \quad P_t = \sum_k \lambda_k v_k u_k^\top.$$

Eigenvalues ordered $\lambda_1 = 1 > |\lambda_2| \geq |\lambda_3| \geq \cdots$ (Perron–Frobenius). The Perron pair: $v_1 = \mathbf{1}$, $u_1 = \mu_t^{\mathrm{PS}}$.

*RL semantics.* The Bellman operator propagates value $V \mapsto \mathbf{r} + \gamma P_t V$ via right action of $P_t$; the convergence rate of the slowest non-trivial spectral mode is $\gamma|\lambda_2|$ and the right eigenvectors $\{v_k\}$ describe the frequency of forward value diffusion. The PS predecessor sweep propagates priority via left action of $P_t^\top$; its convergence is governed by the left eigenvectors $\{u_k\}$. For a non-reversible chain $v_k \not\propto u_k$: the two sweep directions are spectrally distinct. $v_2$ **identifies the slowest forward-diffusion states;** $u_2$ **identifies the slowest backward-attribution states.**

**Graph Fourier Transform interpretation.** The biorthogonal eigendecomposition $P_t = \sum_k \lambda_k v_k u_k^\top$ is the *Graph Fourier Transform* (GFT) on the state-transition graph.

For a reward signal $\mathbf{r}$, the GFT coefficients $\hat{r}_k = u_k^\top \mathbf{r}$ measure the projection onto each spectral mode; the inverse GFT reconstructs $\mathbf{r} = \sum_k \hat{r}_k v_k$. The value function is then $\mathbf{f}^{out} = (L_t^{out} + \varepsilon I)^{-1}\mathbf{r} = \sum_k \frac{\hat{r}_k}{1-\lambda_k+\varepsilon} v_k$ — a *low-pass filter*. Modes with $\lambda_k \approx 1$ (slow-mixing, near positive real axis) are amplified by $\frac{1}{|1-\lambda_k+\varepsilon|} \approx 1/\varepsilon$; modes with $\mathrm{Re}(\lambda_k) \ll 1$ (fast-decaying) are suppressed.

**Dissipative vs. circulatory operator components.** The source/sink and curl language used later should be understood heuristically at the *operator* level. With respect to the stationary inner product $L_2(\mu_t^{\mathrm{PS}})$, write

$$P_t = S_t + A_t, \qquad S_t = \tfrac{1}{2}(P_t + P_t^*), \qquad A_t = \tfrac{1}{2}(P_t - P_t^*),$$

so that

$$L_t^{out} = I - P_t = (I - S_t) - A_t.$$

The self-adjoint part $I - S_t$ is the dissipative/reversible component: it contracts variation in the sense of Laplacian smoothing. The skew-adjoint part $A_t$ is circulatory: it transports mass around directed cycles without directly dissipating it.

**Perturbation theory.** For a localized topology-policy update $\delta\pi_t^{\mathrm{PS}}$ at state $s$, the kernel changes by $\delta P_t = e_s\, w^\top$ where $w = \sum_a \delta\pi_t^{\mathrm{PS}}(s,a)\, p(\cdot|s,a)$ (rank-1, $O(\deg)$ nonzeros).

The first-order eigenvalue perturbation is

$$\delta\lambda_2 \approx \frac{u_2^\top (\delta P_t)\, v_2}{u_2^\top v_2} = \frac{u_2(s)\,(w^\top v_2)}{u_2^\top v_2}, \quad \text{cost } O(\deg).$$

For real $\lambda_2$ (reversible chains), $\delta(\mathrm{SLEM}) = \delta\lambda_2$. For complex $\lambda_2$ (generic directed chains), $\delta(\mathrm{SLEM}) = \mathrm{Re}\big(\frac{\bar{\lambda}_2}{|\lambda_2|}\,\delta\lambda_2\big)$. For the corresponding left/right eigenvectors, standard first-order perturbation theory gives the scaling $\|(\delta v_2, \delta u_2)\| = O\big(\kappa(\lambda_2)\|\delta P_t\|_2/\Delta\lambda_{2,3}\big)$, where $\Delta\lambda_{2,3} = |\lambda_2| - |\lambda_3|$. Thus eigenvector sensitivity becomes large either when the eigengap closes or when the chain is strongly non-normal; no uniform $\kappa(\lambda_2)$ bound is available under the current assumptions.

**SLEM and policy structure.** SLEM measures the persistence of the slowest non-trivial mode of the policy-induced chain. Two structural limits:

- *Cyclic (curl-dominated) flow*: if the optimal policy sustains persistent cycles (e.g., patrol tasks), $P_t$ approaches a permutation matrix. Its non-unit eigenvalues lie on the unit circle, so $\mathrm{SLEM}(P_t) \to 1$ (near-periodic, slow-mixing).
- *Sink-seeking (single-attractor) flow*: if the optimal policy directs most states toward one absorbing goal, then after the softmax-induced stochasticity needed for ergodicity the non-trivial modes are strongly damped, so $\mathrm{SLEM}(P_t)$ can be close to 0. Typical in shortest-path and goal-reaching MDPs.

These are informative limiting cases, not a general monotone law. Large SLEM can arise from near-periodic cycling, bottlenecks, or metastable basins; low SLEM indicates rapid contraction, not "more divergence" per se.

**Computational Methods**

**Construction of $L_t^{out}$ and $L_t^{in}$.** Given per-action matrices $P^a$ and topology policy $\pi_t^{\mathrm{PS}}$, construct $P_t = \sum_a \mathrm{diag}(\pi_t^{\mathrm{PS}}(\cdot, a))P^a$ in $O(|\mathcal{S}| \cdot |\mathcal{A}| \cdot \deg)$.

$$L_t^{out} = I - P_t, \qquad D_t^{in} = \mathrm{diag}(P_t^\top \mathbf{1}), \quad L_t^{in} = D_t^{in} - P_t^\top.$$

Both $L_t^{out}$ and $L_t^{in}$ are stored in CSR format and inherit the sparsity of $P_t$. $L_t^{in}$ costs one extra transpose, $O(|\mathcal{S}| \cdot \deg)$.

**$\varepsilon$-regularized Laplacian inverse as potential operator.** $L_t^{out} = I - P_t$ is singular (stationary null vector $\mathbf{1}$); adding $\varepsilon I$ shifts all eigenvalues: $\mathrm{Re}(\lambda_k(L_t^{out} + \varepsilon I)) = 1 - \mathrm{Re}(\lambda_k(P_t)) + \varepsilon \geq \varepsilon > 0$ for any ergodic chain, so the inverse is well-defined without any symmetry requirement. The forward and backward potentials are:

$$\mathbf{f}_t^{out} = (L_t^{out} + \varepsilon I)^{-1}\mathbf{r}, \qquad \mathbf{f}_t^{in} = (L_t^{in} + \varepsilon I)^{-1}\mathbf{r}.$$

$\mathbf{f}_t^{out}$ is the reward diffused through the *forward* topology; $\mathbf{f}_t^{in}$ is the reward backpropagated through the *edge-reversed graph* (backward sweep efficiency signal). Both are computed once at initialisation and updated incrementally.

**Full update for $\mathbf{f}^{out}$ and $\mathbf{f}^{in}$.** *Method A: GMRES (Krylov iterative solver).* Since $L_t^{out} + \varepsilon I$ is non-symmetric, Conjugate Gradient does not apply. GMRES (Generalized Minimal RESidual) solves $(L_t^{out} + \varepsilon I)x = b$ by minimizing the residual norm over the Krylov subspace $\mathcal{K}_j(A, b) = \mathrm{span}\{b, Ab, \ldots, A^{j-1}b\}$, requiring no symmetry. Each iteration at step $j$ costs $O(|\mathcal{S}|(\deg + j))$ (one mat-vec plus Gram-Schmidt against $j$ prior vectors); restarted GMRES($m$) costs $O(m|\mathcal{S}|\deg + m^2|\mathcal{S}|)$ per restart and converges in $m = 10$–$20$ restarts to high accuracy in practice. ILU(0) preconditioning reduces the effective spectral spread at $O(|\mathcal{S}| \cdot \deg)$ setup cost.

*Method B: Neumann polynomial (matrix-free approximation).* Write $A = L_t^{out} + \varepsilon I = (1 + \varepsilon)(I - \frac{1}{1+\varepsilon}P_t)$. Since $\rho(P_t) = 1$ (row-stochastic), the resolvent expands as the convergent Neumann series

$$(L_t^{out} + \varepsilon I)^{-1} = \sum_{k=0}^{\infty} \frac{P_t^k}{(1 + \varepsilon)^{k+1}}.$$

The $K$-term truncation $\mathcal{N}_t^{(K)} := \sum_{k=0}^{K} P_t^k/(1 + \varepsilon)^{k+1}$ gives the approximation $\mathbf{f}_t^{out} \approx \mathcal{N}_t^{(K)}\mathbf{r}$, computed as $K$ sequential sparse mat-vecs at cost $O(K|\mathcal{S}|\deg)$. Since $\|P_t^k\|_\infty = 1$ (row-stochastic), the sup-norm truncation error is

$$\left\|\mathbf{f}_t^{out} - \mathcal{N}_t^{(K)}\mathbf{r}\right\|_\infty \leq \frac{\|\mathbf{r}\|_\infty}{\varepsilon(1 + \varepsilon)^{K+1}}.$$

*Note:* GMRES provides no a priori convergence rate for general non-symmetric systems. The classical Eisenstat–Elman–Schultz (EES) residual bound requires the symmetric part $\frac{1}{2}(A + A^\top)$ to be positive definite — equivalently, $\mathrm{Re}(\lambda_k(A)) > 0$ and $A$ nearly normal, which does not hold for $L_t^{out} + \varepsilon I$ when $P_t$ is non-reversible and $\varepsilon$ is small. The Neumann bound is *unconditional*: it requires only $\rho(P_t) = 1$, which holds for any ergodic chain regardless of symmetry or normality. In practice GMRES converges faster and is preferred for high-accuracy solves; the Neumann polynomial is preferred when a certified error bound is needed or when moderate accuracy ($K = 20$–$50$ mat-vecs) is sufficient.

**Incremental for $\mathbf{f}^{out}$ and $\mathbf{f}^{in}$.** The inverse $G_t := (L_t^{out} + \varepsilon I)^{-1}$ is dense in general, so the vector incremental $\delta\mathbf{f}^{out} \approx -G_t(\delta P_t)\mathbf{f}^{out}$ is dense as well. This gives two natural column-cache regimes for $\mathbf{g}_s^{(t)} := G_t e_s$.

*Exact cache.* After the rank-1 update $\delta L_t^{out} = -e_{s_{\mathrm{upd}}} w^\top$, we maintain $G_t$ by Sherman–Morrison; all columns $\mathbf{g}_s^{(t)}$ stay current, but each update costs $O(|\mathcal{S}|^2)$ because the outer product is dense.

*Stale cache.* We keep a previously refreshed column $\mathbf{g}_s^{(\mathrm{ref})}$ and use $\delta\mathbf{f}^{out} \approx (w^\top\mathbf{f}^{out})\,\mathbf{g}_{s_{\mathrm{upd}}}^{(\mathrm{ref})}$ at only $O(|\mathcal{S}|)$ read cost, accepting an error that grows with accumulated policy drift until the next refresh.

For $\mathbf{f}^{in}$, the perturbation $\delta L_t^{in} = \mathrm{diag}(w) - we_{s_{\mathrm{upd}}}^\top$ is not rank-1, so there is no analogous cheap exact cache maintenance:

$$\delta\mathbf{f}^{in} \approx -(L_t^{in} + \varepsilon I)^{-1}\left[w \odot \mathbf{f}^{in} - \mathbf{f}^{in}(s_{\mathrm{upd}}) \cdot w\right]$$

cost $O(|\mathcal{S}| \cdot \deg)$ with $(L_t^{in} + \varepsilon I)^{-1}$ cached.

**Arnoldi for biorthogonal pair $(v_2, u_2)$.** The dominant non-unit eigenpair of $P_t$ corresponds to SLEM. The Arnoldi process builds a $j$-step Krylov basis $\{b, P_t b, P_t^2 b, \ldots\}$ and extracts Ritz pairs in $O(j \cdot |\mathcal{S}| \cdot \deg + j^2|\mathcal{S}|)$ (mat-vec plus Gram-Schmidt; simplifies to $O(j|\mathcal{S}|\deg)$ when $\deg \geq j$). For the biorthogonal pair $(v_2, u_2)$, run Arnoldi once on $P_t$ (for $v_2$) and once on $P_t^\top$ (for $u_2$); $j = 10$–$20$ suffices in practice. First-order eigenvalue tracking between recomputations: $\delta\lambda_2 \approx u_2^\top(\delta P_t)v_2/(u_2^\top v_2)$, cost $O(\deg)$; for complex $\lambda_2$, $\delta(\text{SLEM}) = \text{Re}(\bar{\lambda}_2|\lambda_2|^{-1}\,\delta\lambda_2)$.

**Incremental update chain.** A localized policy change at $s_{\text{upd}}$ gives $\delta P_t = e_{s_{\text{upd}}} w^\top$ (rank-1):

1. $\delta P_t$: rank-1, row $s_{\text{upd}}$ only. Cost: $O(\deg)$.
2. $\delta L_t^{out} = -\delta P_t$; $\delta D_t^{in} = \text{diag}((\delta P_t)^\top \mathbf{1})$; $\delta L_t^{in} = \delta D_t^{in} - (\delta P_t)^\top$. Cost: $O(\deg)$.
3. $\delta\mathbf{f}^{out} \approx (w^\top \mathbf{f}^{out}) \cdot (L_t^{out} + \varepsilon I)^{-1} e_{s_{\text{upd}}}$. Reading $\mathbf{g}_{s_{\text{upd}}}^{(t)} = (L_t^{out} + \varepsilon I)^{-1} e_{s_{\text{upd}}}$ costs $O(|\mathcal{S}|)$ *if the column is current*. However, by Sherman–Morrison each rank-1 inverse update: $G^{(t+1)} = G^{(t)} + \frac{\mathbf{g}_{s_{\text{upd}}}^{(t)} (w^\top G^{(t)})}{1 - w^\top \mathbf{g}_{s_{\text{upd}}}^{(t)}}$ is $O(|\mathcal{S}|^2)$ because of dense $\mathbf{g}_{s_{\text{upd}}}^{(t)}$ and $w^\top G^{(t)}$. Hence maintaining an exact cache is not economical enough, specifically when the drift often accumulates faster than $O(|\mathcal{S}|)$.
4. For $\delta\mathbf{f}^{in}$: since $\delta L_t^{in} = \text{diag}(w) - w e_{s_{\text{upd}}}^\top$ is *not* rank-1,
$$\delta\mathbf{f}^{in} \approx -(L_t^{in} + \varepsilon I)^{-1}\big[w \odot \mathbf{f}^{in} - \mathbf{f}^{in}(s_{\text{upd}}) \cdot w\big];$$
cost $O(|\mathcal{S}| \cdot \deg)$ with $(L_t^{in} + \varepsilon I)^{-1}$ cached (applied to an $O(\deg)$-sparse vector).
5. $\delta\lambda_2 = u_2^\top(\delta P_t)v_2/(u_2^\top v_2) = u_2(s_{\text{upd}})(w^\top v_2)/(u_2^\top v_2)$; $\delta(\text{SLEM}) = \text{Re}(\bar{\lambda}_2|\lambda_2|^{-1}\delta\lambda_2)$ for complex $\lambda_2$. Cost: $O(\deg)$.

When reward shifts by $\delta\mathbf{r}$:

$$\delta\mathbf{f}^{out} = (L_t^{out} + \varepsilon I)^{-1}\delta\mathbf{r}, \qquad \delta\mathbf{f}^{in} = (L_t^{in} + \varepsilon I)^{-1}\delta\mathbf{r}.$$

With precomputed inverse columns for $\text{supp}(\delta\mathbf{r})$, the exact cost is $O(|\text{supp}(\delta\mathbf{r})| \cdot |\mathcal{S}|)$; with only a dense LU factorization, the exact triangular solve is $O(|\mathcal{S}|^2)$ per potential; otherwise use one GMRES solve per potential at $O(m|\mathcal{S}|\deg)$ per restart.

**Method summary.**

| Method | Computes | Cost |
|---|---|---|
| LU decomp. (`dgetrf`) | $(L_t^{out} + \varepsilon I)^{-1}$, cached | $O(\|\mathcal{S}\|^3)$; one-time |
| GMRES($m$) on $L_t^{out} + \varepsilon I$ | $(L_t^{out} + \varepsilon I)^{-1}\mathbf{r}$ (one solve) | $O(m\|\mathcal{S}\|\deg + m^2\|\mathcal{S}\|)$ per restart |
| Neumann poly $\mathcal{N}_t^{(K)}\mathbf{r}$ | resolvent approx, err $\leq \frac{\|\mathbf{r}\|_\infty}{\varepsilon(1+\varepsilon)^{K+1}}$ | $O(K \cdot \|\mathcal{S}\| \cdot \deg)$; no symmetry required |
| Poly. diffusion $(I - \eta L^{out})^k$ | finite-horizon potential | $O(k \cdot \|\mathcal{S}\| \cdot \deg)$ |
| Arnoldi ($j$ steps, $P_t$) | $(\lambda_2, v_2)$ | $O(j \cdot \|\mathcal{S}\| \cdot \deg + j^2\|\mathcal{S}\|)$ |
| Arnoldi ($j$ steps, $P_t^\top$) | $u_2$ | same |
| Incr. update $\delta L_t^{out}, \delta L_t^{in}$ | from $\delta P_t$ | $O(\deg)$ |
| Incr. update $\delta\mathbf{f}^{out}$ | $(w^\top \mathbf{f}^{out}) \cdot \mathbf{g}_s$ | $O(\|\mathcal{S}\|^2)$ exact (SM update) or $O(\|\mathcal{S}\|)$ stale approx |
| Incr. update $\delta\mathbf{f}^{in}$ | $-(L^{in} + \varepsilon I)^{-1}[w \odot \mathbf{f}^{in} - \mathbf{f}^{in}(s)w]$ | $O(\|\mathcal{S}\| \cdot \deg)$ with $(L^{in} + \varepsilon I)^{-1}$ cached |
| $\delta\lambda_2$ tracking (1st-order) | $u_2^\top(\delta P_t)v_2/(u_2^\top v_2)$ | $O(\deg)$ |
| SLEM recompute (Arnoldi, lazy) | $(\text{SLEM}, v_2, u_2)$ exact | $O(j \cdot \|\mathcal{S}\| \cdot \deg)$ every $T_{\text{arn}}$ steps |

**Augmented Priority Signal**

Standard PS priority $|\xi(s)| = |\Xi V(s) - V(s)|$ (Bellman residual) remains negligible until the Bellman error wave reaches $s$ via predecessor propagation — purely reactive. GTA-PS augments it with a proactive *directional topology signal* derived from both potential functions.

**Forward and backward potentials.**

$$\mathbf{f}_t^{out} = (L_t^{out} + \varepsilon I)^{-1}\mathbf{r}_t, \qquad \mathbf{f}_t^{in} = (L_t^{in} + \varepsilon I)^{-1}\mathbf{r}_t.$$

$\mathbf{f}_t^{out}(s)$ measures the expected future reward *accessible from* $s$ via the forward topology — equivalently, the value of $s$ under $\pi_t^{\mathrm{PS}}$ (the Neumann series sums $P_t^k\mathbf{r}$ at row $s$, propagating forward from $s$). $\mathbf{f}_t^{in}(s)$ measures reward that flows *into* $s$ from predecessors via the edge-reversed graph (backward sweep signal).

**Priority formula.** Define the reactive Bellman-residual operator

$$\xi_t(s) \triangleq |\Xi V_t(s) - V_t(s)|,$$

and the topology-only ranking term

$$\tau_t(s) \triangleq (1-\alpha_t)\,|\delta\mathbf{f}^{out}(s)| + \alpha_t\,|\delta\mathbf{f}^{in}(s)|,$$

Here $\delta\mathbf{f}^{out} = \mathbf{f}_{t+1}^{out} - \mathbf{f}_t^{out}$ (and likewise for $\mathbf{f}^{in}$).

The aggregated queue key used by GTA-PS is:

$$\mathrm{priority}_t(s) \triangleq \xi_t(s) + \beta_t\,\tau_t(s).$$

The Bellman residual $\xi_t(s)$ remains the reactive base signal; $\tau_t(s)$ is the topology-induced correction that reorders states when the geometry predicts future propagation.

In a fixed-mixing baseline, $\alpha_t$ may be held constant. $\beta_t$ generally need annealing for stability, but in principle could be held constant if the topology signal is well-scaled with the Bellman error. Section 7 defines the SLEM-driven polynomial schedule used by GTA-PS$^{\alpha}$ and linear schedule used by GTA-PS$^{\beta}$.

*Policy-induced change* ($\mathbf{r}$ fixed, $P_t \to P_{t+1}$):

$$\delta\mathbf{f}^{out} \approx (w^\top \mathbf{f}_t^{out}) \cdot (L_t^{out} + \varepsilon I)^{-1} e_s,$$
$$\delta\mathbf{f}^{in} \approx -(L_t^{in} + \varepsilon I)^{-1}\big[w \odot \mathbf{f}_t^{in} - \mathbf{f}_t^{in}(s)\cdot w\big],$$

*Reward shift* ($P_t$ fixed, $\mathbf{r} \to \mathbf{r} + \delta\mathbf{r}$):

$$\delta\mathbf{f}^{out} = (L_t^{out} + \varepsilon I)^{-1}\delta\mathbf{r}, \qquad \delta\mathbf{f}^{in} = (L_t^{in} + \varepsilon I)^{-1}\,\delta\mathbf{r},$$

States at the boundary of the changed reward region receive large $|\delta\mathbf{f}^{out}|$ and $|\delta\mathbf{f}^{in}|$ before any Bellman error has propagated, providing the proactive anticipation that distinguishes GTA-PS from standard PS.

**Two-queue implementation and top-$q$ filtering.** Because $\tau_t(\cdot)$ is generically dense after either a reward shift or a topology-policy update, inserting the full topology field directly into the same heap as the sparse residual field would crowd the queue and force frequent global rebuilds. The practical implementation therefore keeps two persistent queues:

$$\mathcal{Q}_\xi(t) \text{ over the active residual set } \mathcal{I}_t \triangleq \{s : \xi_t(s) > 0\}, \qquad \mathcal{Q}_\tau(t) \text{ over all states keyed by } \tau_t(s).$$

After each dense refresh of $\tau_t$, the module materializes a filtered topology frontier

$$\mathcal{F}_q(t) \triangleq \textsc{TopQ}(\mathcal{Q}_\tau(t), q_t), \qquad K \le q_t \le |\mathcal{S}|,$$

and stores it as a small frontier heap $\mathcal{H}_q(t)$ keyed by the aggregate score $\xi_t(s) + \beta_t\tau_t(s)$ for $s \in \mathcal{F}_q(t)$.

The actual backup candidate set is then

$$\mathcal{C}_t \triangleq \mathrm{Keys}(\mathcal{Q}_\xi(t)) \cup \mathcal{F}_q(t),$$

and the module pops

$$s_t = \arg\max_{s\in\mathcal{C}_t}\big\{\xi_t(s) + \beta_t\tau_t(s)\big\}.$$

This separation preserves the intended roles of the two operators: $\mathcal{Q}_\xi$ guarantees that states with large local Bellman inconsistency remain eligible even when their topology score is modest, while $\mathcal{F}_q$ injects only the most topology-relevant states instead of flooding the queue with a globally nonzero $\tau_t$ vector.

Setting $q_t = |\mathcal{S}|$ recovers the unfiltered algorithm analyzed in Section 7. Choosing $q_t \ll |\mathcal{S}|$ is a practical approximation that controls queue crowding and keeps the dense topology refresh at $O(|\mathcal{S}| + q_t \log|\mathcal{S}|)$ instead of turning every planning step into a full dense candidate scan. The finite-$K$ bound in Proposition 7.8 therefore applies exactly to the unfiltered case $q_t = |\mathcal{S}|$; the filtered version additionally requires that the true top-$q$ aggregate priorities lie inside $\mathcal{C}_t$.

## 7.1 Preliminary

**Structure.** The section is organized through six core layers, each building on the previous.

- **§7.1** (operator theory) establishes resolvent norm bounds and the Green's function spectral representation of the priority signal.
- **§7.2** (sensitivity) derives first-order perturbation formulas for SLEM and the resolvent under rank-1 policy updates.
- **§7.3** (propagation lower bounds) formalizes the mean first-passage time barrier of reactive PS and GTA-PS.
- **§7.4** (behavior analysis) develops the GTA-PS residual trajectory, the budget-residual dominance criterion, the exact topology-policy signal at $\alpha_t = 0$, and the forward decision-oracle mismatch bound.
- **§7.5** (smoothness analysis) develops the energy-based smoothness bound for the GTA-PS priority signal, as well as the smoothness of perturbation-induced priority changes.
- **§7.6** (decision analysis) develops the theoretical guarantees for the GTA-PS decision rule, comparing it to the idealized decision rule that leads to the best possible discrepancy reduction at K steps.

**Roadmap.** The roadmap of the analysis is as follows. Sections 7.1–7.3 provide the supporting machinery: operator theory gives the resolvent / Green's-function representation of the topology signal, sensitivity analysis gives the first-order response of both the resolvent and SLEM under local policy updates, and the MFPT barrier formalizes why reactive PS misses distant predecessor states after a reward shift. The analytical center is then split into three perspectives.

- *Behavior analysis* (Section 7.4) studies the polar / asymptotic regimes of the policy-induced chain and how $(\mathrm{SLEM}, A_t, b, \alpha_t, \beta_t)$ govern the exploration–exploitation allocation of the topology signal.
- *Smoothness analysis* (Section 7.5) compares the known reward-shift landscape $\delta\mathbf{r}$ with its diffused images $\delta\mathbf{f}^{out}$, $\delta\mathbf{f}^{in}$, and $\boldsymbol{\tau}$ through density, discrepancy, and energy-based regularity.
- *Decision analysis* (Section 7.6) studies the induced ranking rule itself: top-1 identification, top-$q$ coverage, and the accuracy–coverage tradeoff created by forward/backward mixing.

**Notation.** We distinguish two oracle notions throughout the theoretical analysis. First, the *greedy action oracle* is the structural object induced by the host advantages: at time $t$, let $a_t^\sharp(s) \in \arg\max_a A_t(s,a)$ and let its induced one-step greedy kernel be $P_t^\sharp(s,s') \triangleq p(s'|s, a_t^\sharp(s))$; Section 7.6 compares the topology score to the corresponding value-perturbation landscape $\psi_t(s) = \gamma\delta V_t^\sharp(s)$. Second, for actual backup selection at planning step $k$, we use a *feasible ranking oracle*. Let $V_{t,k}$ be the host value estimate after $k$ backups in the current replanning episode, let $B_s^P$ denote a single backup at state $s$ performed under model $P$, and let $D_{t,k}(\cdot)$ be the discrepancy objective being optimized (for example $\|V - V_t^*\|_\infty$ in the known-model analysis, or a model-internal discrepancy under learned $\hat{p}_t$ in Dyna). The one-step oracle gain under model $P$ is

$$g_{t,k}^P(s) \triangleq D_{t,k}(V_{t,k}) - D_{t,k}(B_s^P V_{t,k}).$$

The top-1 feasible oracle is $s_{t,k}^{\star,P} \in \arg\max_s g_{t,k}^P(s)$, and the top-$q$ feasible ranking oracle $\mathcal{S}_{t,k}^{\star,q,P}$ is obtained by sorting the gains $g_{t,k}^P(s)$ and taking the $q$ largest states. When $P = p$, this is a retrospective true oracle for the known-model setting; when $P = \hat{p}_t$, it is the information-constrained model oracle appropriate for Dyna. By contrast, the exact *schedule oracle* over ordered $q$-backup sequences is generally combinatorial because backup effects interact across steps; the memo treats that object only as a conceptual upper bound, not as a tractable benchmark.

### Sign, Mixing, and GTA-PS Variants

**Sign of the priority signal.** $\delta\mathbf{f}^{out}(s) > 0$ when the forward potential at $s$ increases after a policy or reward update — the agent's expected discounted reward from $s$ has grown, making $s$ exploit-relevant. $\delta\mathbf{f}^{out}(s) < 0$ signals states where potential has fallen — correction-relevant under the current topology. Because the priority formula uses absolute values, the topology term measures the *magnitude* of the structural shift; the sign is still diagnostically useful, but $\beta_t < 0$ acts as a topology dampener rather than a sign-selective switch.

**GTA-PS$^\alpha$ mixing rule.** By convention $\alpha_t$ weights the backward signal: $\alpha_t \to 0$ gives strong forward priority, while $\alpha_t \to 1$ gives heavier backward priority. The **GTA-PS$^\alpha$** variant sets this backward weight by a polynomial schedule in $\mathrm{SLEM}(P_t)$:

$$\alpha_t = \mathrm{SLEM}(P_t)^\alpha, \qquad \alpha > 0.$$

Since SLEM $\in [0, 1)$ for an ergodic chain, this guarantees $\alpha_t \in [0, 1)$. When mixing is fast (small SLEM), $\alpha_t \approx 0$ so the forward signal dominates. When the induced GTA chain develops persistent slow modes (for example from near-periodicity, multi-attractor competition, or bottlenecks), SLEM$(P_t)$ rises and the weight shifts monotonically toward the backward signal. The exponent $\alpha$ controls the handover rate:

- $\alpha = 1$: linear-in-SLEM baseline.
- $\alpha > 1$: delayed transfer to backward priority (forward dominates longer).
- $0 < \alpha < 1$: accelerated transfer to backward priority early in training.

Equivalently, $\alpha_t$ is strictly increasing in SLEM$(P_t)$, with slope $\alpha\,\mathrm{SLEM}(P_t)^{\alpha-1} > 0$, so the schedule is monotone and parametrically tunable throughout training. In the present memo this schedule is exploratory: it is useful for ablation studies on forward/backward mixing, but the core theory remains anchored at $\alpha_t = 0$. Over training, $\alpha_t$ need not move monotonically in time: it increases or decreases with whatever geometric changes the current $\pi_t^{\mathrm{PS}}$ induces. Exploration/exploitation language is therefore only heuristic: a more exploratory $\pi_t^{\mathrm{PS}}$ often preserves more curl and can raise SLEM, but this correlation is task-dependent and environment-dependent rather than a theorem.

**GTA-PS$^\beta$ schedule.** The **GTA-PS$^\beta$** variant proposes a SLEM-coupled linear schedule for the directed setting using the directed spectral gap $1 - \mathrm{SLEM}(P_t)$ as the scheduling signal:

$$\beta_t = \beta_1\big((1 - \mathrm{SLEM}(P_t)) - \beta_2\big),$$

with hyperparameters $\beta_1 > 0$ (gain) and $\beta_2 \in (0, 1)$ (crossover threshold).

- When $1 - \mathrm{SLEM}(P_t) > \beta_2$: $\beta_t > 0 \to$ topology signal amplified; the induced GTA chain is in a relatively fast-contracting regime and the topology cue is treated as more reliable.
- When $1 - \mathrm{SLEM}(P_t) < \beta_2$: $\beta_t < 0 \to$ topology signal dampened; the induced GTA chain has persistent slow modes, so topology is treated more cautiously.

Thus $\beta_t$ should be read primarily as a geometry-confidence schedule for the current GTA-induced flow, with exploration/exploitation effects appearing only insofar as they correlate with that geometry through $\pi_t^{\mathrm{PS}}$. It is a heuristic scheduler rather than a standalone theoretical guarantee. As SLEM$(P_t) \to 1$ (induced flow becomes strongly cyclic / near-periodic), $\beta_t \to -\beta_1\beta_2$ (fixed damping floor).

**GTA-PS$^{\alpha,\beta}$ coupling.** The **GTA-PS$^{\alpha,\beta}$** variant combines both SLEM-coupled mechanisms: the polynomial mixing weight $\alpha_t = \mathrm{SLEM}(P_t)^\alpha$ (GTA-PS$^\alpha$) and the adaptive topology weight schedule $\beta_t$ (GTA-PS$^\beta$). The resulting priority formula

$$\mathrm{priority}(s) = |\xi(s)| + \beta_t\big[(1 - \alpha_t)|\delta f^{out}(s)| + \alpha_t|\delta f^{in}(s)|\big]$$

is fully driven by the policy-induced geometry summarized by SLEM: both the forward/backward balance ($\alpha_t$) and the topology weight magnitude ($\beta_t$) are derived from a single spectral quantity computed at negligible incremental cost. Exploration/exploitation effects may correlate with this geometry through $\pi_t^{\mathrm{PS}}$, but they are secondary to the scheduler's geometric meaning. This enables ablation studies comparing GTA-PS (fixed $\alpha$, fixed $\beta$), GTA-PS$^\alpha$ (self-tuning $\alpha$, fixed $\beta$), GTA-PS$^\beta$ (fixed $\alpha$, SLEM-scheduled $\beta$), and GTA-PS$^{\alpha,\beta}$ (both coupled). In the main memo, these variants should be read as scheduler ablations rather than as separate theorem-closed algorithms.

**Potential Operator Choice**

The memo's core operator is the $\varepsilon$-resolvent $(L_t^{out} + \varepsilon I)^{-1}$, together with its backward analogue $(L_t^{in} + \varepsilon I)^{-1}$. This is the only operator family used in the main theoretical development, because it aligns exactly with $\gamma V^{\pi_t^{\mathrm{PS}}}$ at $\varepsilon^* = (1 - \gamma)/\gamma$ and supports the rank-1 perturbation analysis above. Alternative diffusion surrogates and finite-horizon operator families are deferred to the demoted note.

**Advantages over Model-Based Topology-Based Planning**

The direct competitive landscape for GTA-PS consists of model-based methods that use some form of graph structure to guide backup ordering or signal propagation: standard prioritized sweeping (Moore & Atkeson 1993), topological value iteration (Dai et al. 2011), and spectral predecessor representations (Dayan 1993; Mahadevan & Maggioni 2007). Four structural advantages distinguish GTA-PS from all of them.

*(i) Reactive vs. Proactive Priority [vs. Standard PS / VaST].* Standard PS priority $|(\mathbf{r} + \gamma V)(s) - V(s)|$ is zero for any state $s$ not yet reached by the Bellman wave after a reward shift. Under budget $K \ll |\mathcal{S}|^2|\mathcal{A}|$, distant states from the reward change are never reached. GTA-PS augments the same priority queue with the proactive signal $|\delta\mathbf{f}^{out}(s)|$ and $|\delta\mathbf{f}^{in}(s)|$: these are nonzero at all states as soon as $(L_t^{out} + \varepsilon I)^{-1}\delta\mathbf{r}$ is computed (one solve),

pre-identifying high-impact states before the Bellman wave arrives. VaST (Corneil et al. 2018) faces the same reactive bottleneck even with its learned abstract model, because its backup order remains residual-driven.

*(ii) SCC-Batch vs. All-Horizon Resolvent Priority [vs. TVI/FTVI].* TVI (Dai et al. 2011) uses the hard combinatorial topology of SCCs: it solves each SCC to full convergence in reverse topological DAG order. This is not budget-interruptible and gives no mechanism to re-prioritize after a reward shift (the SCC decomposition depends only on $p$, not $r_t$). GTA-PS uses a continuous spectral topology: the $\varepsilon$-resolvent $(L_t^{out} + \varepsilon I)^{-1}$ integrates *all* horizons simultaneously with $\varepsilon$-weighted attenuation, and the priority queue is per-step interruptible at $O(\log |\mathcal{S}|)$ per backup. The potential update $\delta \mathbf{f}^{out} = (L_t^{out} + \varepsilon I)^{-1} \delta \mathbf{r}$ reweights all state priorities after a reward shift without recomputing any SCCs.

*(iii) Static Recompute vs. Incremental Update [vs. Successor Representation].* The Successor Representation (Dayan 1993) factorises the value function as $V^\pi = M^\pi \mathbf{r}$ where $M^\pi = (I - \gamma P_t)^{-1}$, making reward updates cheap once $M^\pi$ is fixed. However, a policy change requires recomputing $M^\pi$ at $O(|\mathcal{S}|^3)$ cost; this is the bottleneck in any nonstationary setting where $\pi_t$ evolves. GTA-PS maintains $\mathbf{f}_t^{out} = (L_t^{out} + \varepsilon I)^{-1} \mathbf{r}$ with the closed-form incremental formula

$$\delta \mathbf{f}^{out} \approx (w^\top \mathbf{f}^{out}) \cdot (L_t^{out} + \varepsilon I)^{-1} e_s$$

at cost $O(|\mathcal{S}|)$ per policy step (read cached column $\mathbf{g}_{s_{\text{upd}}}$). Reward shifts cost $O(|\text{supp}(\delta \mathbf{r})| \cdot |\mathcal{S}|)$ with the column cache, matching the SR's reward update but also covering policy shifts. Both update types are separable (additive to first order).

*(iv) Undirected/Symmetric vs. Directional Priority [vs. Proto-Value Functions].* Proto-value functions (Mahadevan & Maggioni 2007) use the *symmetric* Laplacian of the undirected state-transition graph, yielding a single orthonormal eigenbasis. This collapses the forward Bellman direction and the backward PS sweep direction into one spectral signal and discards all asymmetric flow information in $P_t$. TVI similarly treats all edges of the transition graph symmetrically within each SCC. GTA-PS retains the directed structure: $\mathbf{f}^{out}$ from $L_t^{out}$ captures forward reward diffusion (Bellman convergence direction), and $\mathbf{f}^{in}$ from $L_t^{in}$ captures backward attribution along the reversed chain (PS sweep efficiency). For a non-reversible $P_t$ the two are spectrally distinct ($v_k \not\propto u_k$), and the mixing parameter $\alpha$ allocates backup effort between them. This directional decomposition has no analogue in any symmetric spectral method.

### Why Nonstationary Reward Is the Right Setting

**Stationary reward eliminates the advantage.** With fixed $r$, standard PS runs to convergence. At convergence $\pi_t^{\text{PS}}$ stabilizes, so $\mathbf{f}_t^{out} = (L_t^{out} + \varepsilon I)^{-1} \mathbf{r}$ is constant and $\delta \mathbf{f}^{out} = 0$ — the GTA-PS topology signal vanishes identically. For deterministic shortest-path problems (sparse $r$, deterministic $p$), Dijkstra's algorithm computes the optimal policy in $O(|\mathcal{S}| \log |\mathcal{S}| + |E|)$ — strictly superior. GTA-PS contributes nothing in either case.

**Nonstationary reward — the setting where GTA-PS dominates.** When $r_t$ shifts at frequency $1/T$, the agent has $T$ steps to replan before the next shift. A full PS sweep costs $O(|\mathcal{S}|^2 |\mathcal{A}|)$; in the budget-constrained regime $T \ll |\mathcal{S}|^2 |\mathcal{A}|$, only $K = O(T/(|\mathcal{A}||\mathcal{S}|))$ Bellman backups are affordable per period.

GTA-PS concentrates these $K$ backups on the states with the largest predicted value change. By Proposition (Section 6), setting $\varepsilon = \varepsilon^*$ (Definition 6) gives $\mathbf{f}_t^{out} = \gamma V^{\pi_t^{\text{PS}}}$, so the priority signal $|\delta \mathbf{f}^{out}(s)| = \gamma |\delta V^{\pi_t^{\text{PS}}}(s)|$ is then exactly proportional to the first-order value change at $s$ — not a heuristic proxy but a direct predictor of Bellman error magnitude. Reactive PS must wait for the Bellman wave to diffuse hop-by-hop before it can assign correct priorities; GTA-PS concentrates backups from the first step after the reward shift.

The SLEM-based GTA-PS$^\alpha$/GTA-PS$^\beta$ schedule additionally adapts to the current geometry of the GTA-induced chain $P_t^{\text{PS}}$, providing automatic topology-aware reweighting that may correlate with exploration/exploitation in some tasks. This regime — known $p$, nonstationary $r_t$, limited replanning budget — characterises logistics scheduling, adaptive robot navigation, and reconfigurable routing.

### Algorithm and Complexity

GTA-PS is a *computation management module*, not a standalone RL algorithm. It maintains: (i) the residual array $\xi[s]$ and a residual queue $\mathcal{Q}_\xi$ over the active residual set $\mathcal{I}_t = \{s : \xi[s] > 0\}$; (ii) the topology array $\tau[s]$, a dense topology queue $\mathcal{Q}_\tau$ over all states, and a filtered frontier $\mathcal{F}_q = \textsc{TopQ}(\mathcal{Q}_\tau, q_t)$ stored as a small heap $\mathcal{H}_q$; (iii) the topology state ($\mathbf{f}^{out}$, $\mathbf{f}^{in}$, SLEM, $v_2$, $u_2$, $\mathbf{g}_s$); and (iv) the topology policy $\pi_t^{\text{PS}}$ (Assumption 6). Decoupling $(\xi[\cdot], \tau[\cdot])$ means the scheduler $(\alpha_t, \beta_t)$ can change without invalidating the underlying signals — only the aggregate score $\xi[s] + \beta_t \tau[s]$ used inside the candidate set must be refreshed. The actual backup candidates are always $\text{Keys}(\mathcal{Q}_\xi) \cup \mathcal{F}_q$; setting $q_t = |\mathcal{S}|$ recovers the original unfiltered implementation.

It exposes GtaPolicyUpdate, PopAndPrioritize, and PushPredecessors calls; the host algorithm performs Bellman backups, computes the current state-local advantage row $A^{\text{PS}}(s, \cdot)$, and controls action selection via its own planning policy $\pi_t$.

**Practical implementation notes.**

- Store $P^a$, $L_t^{out}$, $L_t^{in}$ in CSR format; sparsity inherited from $P_t$.
- Realize $(L_t^{out} + \varepsilon I)^{-1}$ either by exact factorization / Sherman–Morrison maintenance or by sparse iterative approximation; the appropriate choice depends on memory budget and available parallel hardware rather than on a fixed state-count threshold.
- The column cache $\mathbf{g}_s = (L_t^{out} + \varepsilon I)^{-1} e_s$ may be maintained exactly, refreshed lazily, or recomputed on demand. This is an implementation tradeoff between update accuracy, memory traffic, and solver throughput, not a separate theoretical branch.
- Use two binary heaps: $\mathcal{Q}_\xi$ for active residual states and $\mathcal{Q}_\tau$ for the dense topology field. $\mathcal{Q}_\xi$ stores only states whose Bellman residual has been touched, while $\mathcal{Q}_\tau$ is rebuilt from $\tau[\cdot]$ after each dense reward/topology refresh. Then call RefreshTopQ($q_t$) to materialize $\mathcal{F}_q$ and its frontier heap $\mathcal{H}_q$ from the top-$q_t$ entries of $\mathcal{Q}_\tau$. This prevents the dense topology vector from flooding the backup queue while preserving exact residual activation.
- Because the aggregate score depends on both operators, every dense refresh of $\tau[\cdot]$ is followed by RekeyResidualQueue(), which rebuilds $\mathcal{Q}_\xi$ over the active set under the current scores $\xi[s] + \beta_t \tau[s]$. Between dense refreshes, only locally touched predecessor states require $O(\log |\mathcal{S}|)$ residual-queue updates.
- Recompute $(v_2, u_2, \text{SLEM})$ via Arnoldi every $T_{\text{arn}} = \lceil |\mathcal{S}|^{1/2} \rceil$ steps with warm-start; track first-order between recomputations.

**Procedure complexity.**

| Procedure | Cost | When called |
|---|---|---|
| Precompute | $O(\|\mathcal{S}\|^3)$ | once |
| GtaPolicyUpdate | $O(\|\mathcal{A}\|)$ | per backed-up state |
| TopologyUpdate | $O(\|\mathcal{S}\|^2)$ exact / $O(\|\mathcal{S}\| \cdot \deg)$ approx | per policy step |
| $\delta P$, $\delta L^{out}$, $\delta L^{in}$ | $O(\deg)$ | rank-1 matvec |
| SM cache update (exact) | $O(\|\mathcal{S}\|^2)$ | rank-1 outer product (skip for stale approx) |
| $\delta \mathbf{f}^{out}$ (read $\mathbf{g}_s$) | $O(\|\mathcal{S}\|)$ | after SM update (stale $O(\|\text{drift}\|)$ err) |
| $\delta \mathbf{f}^{in}$ (cached $(L_t^{in} + \varepsilon I)^{-1}$) | $O(\|\mathcal{S}\| \cdot \deg)$ | non-rank-1 update |
| $\delta \lambda_2$ tracking | $O(\deg)$ | same; adds $/(u_2^\top v_2)$ scalar |
| Arnoldi recompute (lazy) | $O(j\|\mathcal{S}\| \cdot \deg + j^2 \|\mathcal{S}\|)$, $j = 10$–$20$ | every $T_{\text{arn}}$ |
| Rebuild $\mathcal{Q}_\tau$ + RefreshTopQ | $O(\|\mathcal{S}\| + q_t \log \|\mathcal{S}\|)$ | dense topology refresh |
| RekeyResidualQueue | $O(\|\mathcal{I}_t\| \log \|\mathcal{S}\|)$ | active residual states only |
| RewardUpdate | $O((\|\text{supp}(\delta \mathbf{r})\| + 1) \cdot \|\mathcal{S}\| + (q_t + \|\mathcal{I}_t\|) \log \|\mathcal{S}\|)$ | per shift |
| PopAndPrioritize | $O(\log \|\mathcal{S}\| + \log q_t)$ | per backup |
| PushPredecessors | $O(\|\text{Pred}(s)\| \cdot \log \|\mathcal{S}\|)$ | per backup |

## GTA-PS in Model-based RL algorithms

**Complexity summary.** The dominant GTA-PS work per planning iteration comes from four layers: host Bellman/TD backups, row-local topology-policy updates, dense topology refreshes of $\mathbf{f}^{out}$ and $\mathbf{f}^{in}$ together with the filtered frontier, and periodic Arnoldi recomputation or first-order SLEM tracking. If $C_{\text{pot}}$ denotes the cost of the chosen resolvent update routine, then one dense topology refresh requires

$$O\big(C_{\text{pot}} + q_t \log |\mathcal{S}| + |\mathcal{I}_t| \log |\mathcal{S}| + \deg\big)$$

up to lower-order row-softmax terms, while each host backup retains its underlying $O(|A| \cdot \deg)$ or TD-style cost. These are algorithmic work estimates rather than wall-clock predictions.

Dense vector updates and sparse mat-vecs admit batching and parallel execution, whereas queue operations and row-local policy changes are lighter control overhead. Actual runtime therefore depends strongly on implementation choices such as thread-level parallelism, sparse-kernel fusion, and GPU execution. We avoid finer serialized regime claims and hardware-specific crossover thresholds.

## Experiment Methodology

**Research Questions.** Six falsifiable questions, grounded in the known-model nonstationary setting of Section 1, the ergodic topology-policy assumptions of Section 2, the MFPT / resolvent / reversibility analysis of Section 6, and the adaptive schedules of Section 7.

**Algorithm 1** GTA-PS Module

**Precompute** (once, $O(|\mathcal{S}|^3)$) ▷ $P_0^{\mathrm{PS}} = \sum_a \mathrm{diag}(\pi_0^{\mathrm{PS}}(\cdot, a))\, P^a$
$L_0^{out} \leftarrow I - P_0^{\mathrm{PS}}$; $L_0^{in} \leftarrow D_0^{in} - (P_0^{\mathrm{PS}})^\top$
$(L_0^{out}+\varepsilon I)^{-1}$ via LU; $\mathbf{f}^{out} \leftarrow (L_0^{out} + \varepsilon I)^{-1}\mathbf{r}$; $\mathbf{f}^{in} \leftarrow (L_0^{in} + \varepsilon I)^{-1}\mathbf{r}$
**for each** $s \in \mathcal{S}$: $\mathbf{g}_s \leftarrow (L_0^{out} + \varepsilon I)^{-1} e_s$ ▷ column cache at $t$=0, $O(|\mathcal{S}|^2)$ memory
*Notation:* SLEM below abbreviates SLEM($P_t^{\mathrm{PS}}$).
$(v_2, u_2, \mathrm{SLEM}) \leftarrow \textsc{Arnoldi}(P_0^{\mathrm{PS}}, (P_0^{\mathrm{PS}})^\top)$
$\beta_0 \leftarrow \beta_1((1 - \mathrm{SLEM}) - \beta_2)$; $\alpha_0 \leftarrow \mathrm{SLEM}^\alpha$; choose $q_0 \in [K, |\mathcal{S}|]$
**for each** $s \in \mathcal{S}$: $\xi[s] \leftarrow 0$; $\tau[s] \leftarrow 0$; $\mathcal{Q}_\tau.\textsc{Push}(s,\ 0)$
$\mathcal{Q}_\xi \leftarrow \emptyset$; $\textsc{RefreshTopQ}(q_0)$

**procedure** $\textsc{GtaPolicyUpdate}(s, A^{\mathrm{PS}}(s,\cdot))$ ▷ $O(|\mathcal{A}|)$ row-softmax update
  $\delta\pi^{\mathrm{PS}}(s,\cdot) \leftarrow \mathrm{softmax}(b\, A^{\mathrm{PS}}(s,\cdot)) - \pi^{\mathrm{PS}}(s,\cdot)$
  $\pi^{\mathrm{PS}}(s,\cdot)$ += $\delta\pi^{\mathrm{PS}}(s,\cdot)$ ▷ only row $s$ changes, preserving rank-1 $\delta P_t$
  **return** $\delta\pi^{\mathrm{PS}}(s,\cdot)$
**end procedure**

**procedure** $\textsc{TopologyUpdate}(s_{\mathrm{upd}}, \delta\pi^{\mathrm{PS}})$ ▷ $O(|\mathcal{S}| \cdot \deg)$ topology plus filtered queue refresh
  $w \leftarrow \sum_a \delta\pi^{\mathrm{PS}}(a)\, p(\cdot|s_{\mathrm{upd}}, a)$; $\delta P^{\mathrm{PS}} \leftarrow e_{s_{\mathrm{upd}}}\, w^\top$ ▷ rank-1, $O(\deg)$
  $G \leftarrow G + \frac{\mathbf{g}_{s_{\mathrm{upd}}}(w^\top G)}{1 - w^\top \mathbf{g}_{s_{\mathrm{upd}}}}$; **update** all $\mathbf{g}_{s'} \leftarrow G e_{s'}$ ▷ SM update: $O(|\mathcal{S}|^2)$ exact; skip if stale cache
  $\delta\mathbf{f}^{out} \leftarrow (w^\top \mathbf{f}^{out}) \cdot \mathbf{g}_{s_{\mathrm{upd}}}$; $\mathbf{f}^{out}$ += $\delta\mathbf{f}^{out}$ ▷ $O(|\mathcal{S}|)$; dense
  $\delta\mathbf{f}^{in} \leftarrow -(L_t^{in} + \varepsilon I)^{-1}[w \odot \mathbf{f}^{in} - \mathbf{f}^{in}(s_{\mathrm{upd}}) \cdot w]$; $\mathbf{f}^{in}$ += $\delta\mathbf{f}^{in}$ ▷ $O(|\mathcal{S}|\cdot\deg)$; dense
  $\delta\lambda_2 \leftarrow u_2(s_{\mathrm{upd}})(w^\top v_2)/(u_2^\top v_2)$; SLEM += $\mathrm{Re}(\bar{\lambda}_2/|\lambda_2| \cdot \delta\lambda_2)$ ▷ $O(\deg)$
  **if** $t \bmod T_{\mathrm{arn}} = 0$: $(v_2, u_2, \mathrm{SLEM}) \leftarrow \textsc{Arnoldi}(P_t^{\mathrm{PS}}, (P_t^{\mathrm{PS}})^\top)$
  $\beta_t \leftarrow \beta_1\big((1 - \mathrm{SLEM}) - \beta_2\big)$; $\alpha_t \leftarrow \mathrm{SLEM}^\alpha$
  **for each** $s \in \mathcal{S}$: $\tau[s]$ += $(1-\alpha_t)|\delta f^{out}(s)| + \alpha_t|\delta f^{in}(s)|$ ▷ update topology component; $O(|\mathcal{S}|)$, dense
  Rebuild $\mathcal{Q}_\tau$ from $\{(s, \tau[s])\}_{s\in\mathcal{S}}$ via bottom-up heapify ▷ $O(|\mathcal{S}|)$ dense refresh
  $\textsc{RefreshTopQ}(q_t)$; $\textsc{RekeyResidualQueue}()$
**end procedure**

**procedure** $\textsc{RefreshTopQ}(q_t)$ ▷ $O(q_t \log |\mathcal{S}|)$ frontier refresh after dense $\tau$ update
  $\mathcal{F}_q \leftarrow \textsc{TopQ}(\mathcal{Q}_\tau, q_t)$
  Build $\mathcal{H}_q$ from $\big\{(s,\ \xi[s] + \beta_t \cdot \tau[s])\big\}_{s\in\mathcal{F}_q}$
**end procedure**

**procedure** $\textsc{RekeyResidualQueue}$ ▷ $O(|\mathcal{I}_t| \log |\mathcal{S}|)$ over active residual states
  Rebuild $\mathcal{Q}_\xi$ from $\big\{(s,\ \xi[s] + \beta_t \cdot \tau[s])\big\}_{s:\xi[s]>0}$
**end procedure**

**procedure** $\textsc{RewardUpdate}(\delta\mathbf{r})$ ▷ per reward shift, dense $\tau$ refresh plus top-$q$ frontier update
  $\mathbf{r}$ += $\delta\mathbf{r}$; $\delta\mathbf{f}^{out} \leftarrow (L_t^{out} + \varepsilon I)^{-1}\delta\mathbf{r}$; $\mathbf{f}^{out}$ += $\delta\mathbf{f}^{out}$
  $\delta\mathbf{f}^{in} \leftarrow (L_t^{in} + \varepsilon I)^{-1}\delta\mathbf{r}$; $\mathbf{f}^{in}$ += $\delta\mathbf{f}^{in}$
  **for each** $s \in \mathcal{S}$: $\tau[s]$ += $(1-\alpha_t)|\delta f^{out}(s)| + \alpha_t|\delta f^{in}(s)|$ ▷ reward shift changes $\tau[s]$ globally
  Rebuild $\mathcal{Q}_\tau$ from $\{(s, \tau[s])\}_{s\in\mathcal{S}}$ via bottom-up heapify
  $\textsc{RefreshTopQ}(q_t)$; $\textsc{RekeyResidualQueue}()$
**end procedure**

**procedure** $\textsc{PopAndPrioritize}$ ▷ $O(\log |\mathcal{S}| + \log q_t)$ after frontier materialization
  Let $s_\xi$ be the head of $\mathcal{Q}_\xi$ if nonempty; let $s_\tau$ be the head of $\mathcal{H}_q$ if nonempty
  **return** whichever of $s_\xi$ or $s_\tau$ has larger current score $\xi[s] + \beta_t \cdot \tau[s]$, removing it from every queue in which it appears
**end procedure**

**procedure** $\textsc{PushPredecessors}(s, \Delta V)$ ▷ $O(|\mathrm{Pred}(s)| \log |\mathcal{S}|)$
  **for** $s' \in \mathrm{Pred}(s)$ **do**
    $\xi[s'] \leftarrow |\delta_{s'}|$; $\mathcal{Q}_\xi.\textsc{Update}(s',\ \xi[s'] + \beta_t \cdot \tau[s'])$ ▷ only residual activation is sparse; $\tau[s']$ is read-only here
  **end for**
**end procedure**

**Algorithm 2** DP + GTA-PS

**Require:** module temperature $b > 0$; $A_0^{\mathrm{PS}}(s,a) \leftarrow \gamma \sum_{s'} p(s'|s,a)\, V_0(s') - \max_{a'} \gamma \sum_{s'} p(s'|s,a')\, V_0(s')$; $\pi_0^{\mathrm{PS}}(a|s) \leftarrow \mathrm{softmax}(b\, A_0^{\mathrm{PS}}(s,\cdot)),\ \forall s$

**while** not converged **do**
  **for** $k = 1$ **to** $K$ **do** ▷ $K$ Bellman sweeps per period
    $s \leftarrow$ PopAndPrioritize()
    $V(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} p(s'|s,a)\, V(s')$ ▷ $O(|A| \cdot \deg)$
    PushPredecessors($s$, $\Delta V(s)$)
  **end for**
  $t \leftarrow t + 1$
  **for** each $s$ backed up this period **do**
    $A^{\mathrm{PS}}(s,a) \leftarrow \gamma \sum_{s'} p(s'|s,a)\, V(s') - \max_{a'} \gamma \sum_{s'} p(s'|s,a')\, V(s')$ for each $a$ ▷ $O(|A| \cdot \deg)$
    $\delta\pi^{\mathrm{PS}}(s,\cdot) \leftarrow$ GtaPolicyUpdate($s$, $A^{\mathrm{PS}}(s,\cdot)$) ▷ $O(|A|)$ softmax row update
    TopologyUpdate($s$, $\delta\pi^{\mathrm{PS}}(s,\cdot)$) ▷ rank-1 $\delta P_t^{\mathrm{PS}} = e_s w^\top$; $\|w\|_2 = O(b|\Delta V|)$
  **end for**
  **if** RewardShifted() **then** RewardUpdate($\delta\mathbf{r}$)
  **end if**
**end while**

**Algorithm 3** Dyna (one-step TD) + GTA-PS

**Require:** module temperature $b > 0$; TD learning rate $\eta_{\mathrm{td}} \in (0,1]$; $A_0^{\mathrm{PS}}(s,a) \leftarrow Q_0(s,a) - \max_{a'} Q_0(s,a')$; $\pi_0^{\mathrm{PS}}(a|s) \leftarrow \mathrm{softmax}(b\, A_0^{\mathrm{PS}}(s,\cdot)),\ \forall s$

**while** not converged **do**
  $(s,a,r,s') \leftarrow$ RealStep() ▷ interact with environment; $a \sim \pi_t$ (host policy)
  Update $\hat{p}(\cdot|s,a)$ from $(s,a,s')$ ▷ model learning; uses $\hat{p}$ in place of $p$
  $\xi[s] \leftarrow r + \gamma \max_{a'} Q(s',a') - Q(s,a)$; $Q(s,a)$ += $\eta_{\mathrm{td}}\, \xi[s]$
  $\mathcal{Q}_\xi$.Update($s$, $\xi[s] + \beta_t \cdot \tau[s]$) ▷ enqueue the real-step Bellman residual
  **for** $k = 1$ **to** $K$ **do** ▷ $K$ simulated steps per real step
    $s \leftarrow$ PopAndPrioritize(); $a \leftarrow \arg\max_{a'} Q(s,a')$
    $(r_{\mathrm{sim}}, s'_{\mathrm{sim}}) \sim \hat{p}(\cdot|s,a)$
    $\xi[s] \leftarrow r_{\mathrm{sim}} + \gamma \max_{a'} Q(s'_{\mathrm{sim}}, a') - Q(s,a)$; $Q(s,a)$ += $\eta_{\mathrm{td}}\, \xi[s]$
    $A^{\mathrm{PS}}(s,a) \leftarrow Q(s,a) - \max_{a'} Q(s,a')$ for each $a$
    $\delta\pi^{\mathrm{PS}}(s,\cdot) \leftarrow$ GtaPolicyUpdate($s$, $A^{\mathrm{PS}}(s,\cdot)$) ▷ $O(|A|)$ softmax row update
    TopologyUpdate($s$, $\delta\pi^{\mathrm{PS}}(s,\cdot)$) ▷ rank-1 $\delta P_t^{\mathrm{PS}}$; $\|w\|_2 = O(b|\eta_{\mathrm{td}}\xi[s]|)$
    PushPredecessors($s, \xi[s]$)
  **end for**
  **if** RewardShifted() **then** RewardUpdate($\delta\mathbf{r}$)
  **end if**
**end while**

1. **(RQ1 — Proactiveness under reward shifts.)** *[Sections 1 and 7.3]*
   In the Section 1 setting, reward shifts are state-based and nonstationary, so reactive PS can only respond after Bellman residuals propagate, whereas GTA-PS produces a global resolvent signal immediately. Does this proactive signal reduce post-shift value error and $K_\varepsilon$ relative to Dyna-PS, especially at small replanning budgets $K$ where MFPT-blocked states are still invisible to reactive propagation?

2. **(RQ2 — Topology allocation under adaptive schedules.)** *[Sections 7.4 and 7]*
   GTA-PS allocates priority mass across the residual term, the forward resolvent signal, and exploratory backward mixing through the schedules $\alpha_t$ and $\beta_t$. When do the adaptive schedules outperform fixed allocations, and how do the operating points $(\alpha_t, \beta_t)$ vary with measured SLEM, shift rate, and policy-induced geometry?

3. **(RQ3 — Convergence vs. propagation tradeoff.)** *[Section 7.4; Prop. 7.8]*
   Proposition 7.8 and the associated ablation design suggest an accuracy–coverage tradeoff: small $\alpha_t$ keeps the ranking close to the forward decision oracle, whereas larger $\alpha_t$ may broaden predecessor-side coverage. How much top-1 / top-$q$ decision accuracy is sacrificed for propagation coverage, and which tradeoff yields the best finite-$K$ replanning performance?

4. **(RQ4 — Symmetric vs. asymmetric environment effects.)** *[Sections 2, 7.4, and 8]*
   Under near-reversible or nearly symmetric induced dynamics, $G_{\varepsilon,t}^{out}$ and $G_{\varepsilon,t}^{in}$ should become similar and the benefit of directional mixing should shrink; under asymmetric dynamics they should separate. Does the

GTA-PS gain over symmetric-Laplacian and forward-only controls grow with measured non-reversibility, and does it collapse in near-symmetric regimes as the two signals empirically converge?

5. **(RQ5 — Bottleneck and SCC geometry.)** *[Sections 7.3, 7.4, and 8]*
   GTA-PS is motivated by soft bottlenecks that remain invisible to hard SCC decompositions yet slow reactive propagation through large MFPT barriers. How do doorway bottlenecks, low conductance, metastable basins, and SCC coarse geometry affect the size and location of the GTA-PS advantage, and does the method remain useful when the environment is a single SCC but still geometrically bottlenecked?

6. **(RQ6 — Mutual curl / divergence analysis.)** *[Section 3; Sections 7.2, 7.4, and 8]*
   The operator split $P_t = S_t + A_t$ separates dissipative source–sink structure from antisymmetric circulation. How do the curl proxy $\|A_t\|$ and the divergence / dissipative proxy $\|I - S_t\|$ interact with left-right eigenvector misalignment, backward-mixing response, and schedule response? Is GTA-PS's directional gain best explained by curl alone, divergence alone, or by their joint geometry?

*Note on geometry measurement.* SLEM, conductance / bottleneck proxies, SCC statistics, $\|u_2 - v_2\|_2$, $\|G^{in}_{\varepsilon,t} - G^{out}_{\varepsilon,t}\|_\infty$, and the operator-split norms of $S_t$ and $A_t$ are measured empirically per environment and reward configuration.

**Environments.** We use four environments, listed in decreasing priority, chosen to jointly cover the six research questions while balancing visual interpretability, robotics relevance, and structural control. MiniGrid-FourRooms and GARNET provide exact discrete kernels; Safety-Gymnasium CarGoal and MuJoCo Reacher are low-dimensional control tasks used in the model-based / learned-model branch, with deterministic nominal dynamics and injected stochastic hindrance.

1. **MiniGrid-FourRooms-v1** (Chevalier-Boisvert et al., 2023; Sutton et al., 1999) ($|\mathcal{S}| \approx 384$, $\gamma = 0.99$). This is the *primary showcase environment*. Four rooms $R_1, \ldots, R_4$ are connected by single-cell doorways, giving clean visual bottlenecks and interpretable reward relocation. Its main audit role is RQ1 (proactive post-shift replanning), RQ2 (schedule adaptation under changing geometry), RQ5 (soft bottlenecks beyond SCC structure), and RQ6 (curl/divergence changes when directional wrappers are added). Optional one-way door or drift wrappers provide explicit asymmetry control for RQ4.

2. **Safety-Gymnasium CarGoal** (Ji et al., 2023) (car-based goal-reaching with hazards; $\gamma = 0.99$). This is the *primary robotics hazard-navigation benchmark*. The nominal car dynamics are deterministic and nonholonomic, while hazard regions and injected wind / rain / slip disturbances create stochastic hindrance without collapsing the control structure into a vision task. Its main audit role is RQ1, RQ2, RQ4, and RQ6: it tests whether GTA-PS remains useful in a control problem with asymmetric flow, moving goals, and safety-critical geometry. The action space is discretized into a macro-action set with at least 10 steering-throttle combinations for the planning module.

3. **MuJoCo Reacher** (Todorov et al., 2012; Towers et al., 2024) (planar robotic reaching; $\gamma = 0.99$). This is the *secondary robotics control benchmark*. The deterministic arm dynamics, joint limits, and workspace geometry make the induced flow naturally asymmetric, while stochastic torque perturbations or drag-like hindrance provide controllable disturbance. Constant goal relocation across workspace anchors makes it especially relevant to RQ1, RQ2, and RQ3, with secondary use for RQ4 through measured forward/backward asymmetry. As with CarGoal, the planning module uses a discretized macro-action set with at least 10 actions.

4. **GARNET** (Archibald et al., 1995) ($|\mathcal{S}| \in \{100, 500, 2000\}$, $|\mathcal{A}| \geq 10$, branching factor controlled). GARNET is the *controlled structural benchmark*. It provides exact tabular control of branching factor, sparsity, stochasticity, periodicity, reversibility, and reward support size. Its main audit role is RQ2, RQ4, RQ5, and RQ6: it is where we isolate structural claims that would be confounded by embodiment details in the robotics tasks. GARNET is not the primary visualization environment, but it is the strongest customization benchmark.

**Reward Configurations and Structural Persistence Spectrum.** The reward function $r_t : \mathcal{S} \to \mathbb{R}$ is *state-based, uneven, and sparse*: only states within designated *active zones* carry nonzero reward, and magnitudes differ across zones. This design is to control the number of simultaneously active reward attractors, which changes the persistence structure of the GTA-PS topology policy $\pi_t^{\mathrm{GTA}}$ and hence its induced SLEM.

*Remark on SLEM and reward distribution.* Whether a single dominant attractor produces lower or higher SLEM than multiple competing attractors depends on the interaction between the environment topology (doorway bottlenecks, transition sparsity) and the Boltzmann policy temperature $b$. For that, tracking SLEM empirically per configuration is regarded, though we expect the more dense goal proximity is the lower SLEM it produces. Moreover, attractor can shift between appealling and adversarial, therefore tests the SLEM-based schedule and the robustness of GTA-PS across.

*FourRooms configurations.* Let $R_1, R_2, R_3, R_4$ denote the four rooms. For each configuration $k \in \{1, 2, 3, 4\}$ (number of simultaneously active rooms), select $k$ rooms as the active set $\mathcal{D}_k \subseteq \{R_1, \ldots, R_4\}$ and assign:

$$r_t(s) = \begin{cases} w_i & \text{if } s = g_i \text{ (goal cell of active room } R_i \in \mathcal{D}_k) \\ 0 & \text{otherwise,} \end{cases} \qquad w_i \sim \text{Uniform}([0.5,\ 1.5]), \quad w_i \text{ independent.}$$

Only one goal cell per active room has nonzero reward (maximally sparse within each room); magnitudes $w_i$ are redrawn at each shift epoch.

| Config | Active rooms | Flow regime (qualitative; SLEM measured empirically) |
|---|---|---|
| $k$=1 | 1 of 4 | One active goal; policy directed through single bottleneck |
| $k$=2 | 2 of 4 | Two goals in different rooms; policy must route across doorways |
| $k$=3 | 3 of 4 | Three goals; reward mostly uniform except one empty room |
| $k$=4 | all 4 | All rooms active; magnitudes unequal, creating partial gradient |

Each room has a Gaussian-shaped reward zone generated uniformly at random within the room; the corridor cells have zero reward. At each shift epoch, $\mathcal{D}_k$ is resampled uniformly from all $\binom{4}{k}$ subsets, after that each mixture is sampled for a reward location, with an uniform reward amplitude from $[0.5, 1.5]$. $k$ is held fixed within each experimental condition.

*Safety-Gymnasium CarGoal configurations.* The goal is relocated among a fixed set of anchor waypoints separated by hazards, narrow passages, and open zones. Between shifts, the disturbance field is selected from a small family such as {calm, directional rain, cyclic gust, slippery patch}. This lets one vary asymmetry and stochastic hindrance independently of the goal-relocation mechanism, which is important for RQ2, RQ4, and RQ6.

*MuJoCo Reacher configurations.* The target is relocated among workspace anchors spanning near-body, mid-range, and boundary-reaching configurations. Disturbance modes include none, mild torque noise, sticky drag, and directional bias in one joint. This produces controlled shifts between easier forward-reaching regimes and more adversarial recovery regimes, which are especially useful for RQ2 and RQ3.

*GARNET.* For GARNET ($|\mathcal{S}| \in \{100, 500, 2000\}$), the active reward support is $|\text{supp}(r_t)| \in \{1, 2, 4, 8\}$ randomly chosen states with $r_t(s) \sim \text{Uniform}([0.5, 1.5])$ and $r_t(s') = 0$ elsewhere. In addition to support-size sweeps, the generator is configured to produce reversible vs. non-reversible, sparse vs. dense, and aperiodic vs. near-periodic families so that RQ4–RQ6 can be tested under controlled structural variation.

**Nonstationary Reward Protocol.** At each *shift epoch* the environment resamples its active goal / reward support and, when relevant, its hindrance mode. Formally, each environment instantiates a shift map

$$(r_t, \mathfrak{d}_t) \mapsto (r_{t+T_{\text{shift}}}, \mathfrak{d}_{t+T_{\text{shift}}}),$$

where $\mathfrak{d}_t$ denotes the disturbance regime (none in the purely tabular baseline, directional rain / gust / torque perturbation in the control environments). For FourRooms and GARNET this reduces to support and amplitude resampling; for CarGoal and Reacher it jointly relocates the goal and may switch the hindrance field.

After each shift the agent has $K$ planning backups before the next shift. Defaults: for MiniGrid-FourRooms and GARNET, $T_{\text{shift}} = 5|\mathcal{S}|$ discrete steps; for CarGoal and Reacher, $T_{\text{shift}}$ is set in control steps so that each rollout contains multiple goal relocations and at least one fast-shift regime with $T_{\text{shift}}/t_{\text{mix}}(1/4) \ll 1$.

**Baselines and Ablations.** All methods share the same host planner (DP with known $p$, or Dyna-Q with learned $\hat{p}$); only the backup-selection rule differs. TVI (Dai et al., 2011) is run only in the stationary-reward control condition ($T_{\text{shift}} = \infty$, single-SCC environment) to confirm the claim (Section 5, point ii) that GTA-PS recovers topological ordering when the SCC decomposition is trivial.

| Method | Priority rule | Topology | Nonstationarity | Interrupt. |
|---|---|---|---|---|
| Dyna-Q | Uniform random | × | × | ✓ |
| Dyna-PS | Bellman residual $\lvert\xi(s)\rvert$ | × | Reactive | ✓ |
| PER-Dyna-Q | TD-error importance weight | × | Reactive | ✓ |
| Dyna-2 | Uniform (bg) + tree search (fg) | Partial | × | ✓ |
| TVI | Reverse SCC topological order | Hard (SCC) | × | × |
| GTA-PS (base) | $\lvert\xi(s)\rvert + \beta\lvert\delta f^{out}\rvert$, fixed $\alpha$, $\beta$ | Directed | Proactive | ✓ |
| GTA-PS$^{\alpha}$ | + SLEM-coupled $\alpha_t$ | Directed | Proactive | ✓ |
| GTA-PS$^{\beta}$ | + SLEM-coupled $\beta_t$ | Directed | Proactive | ✓ |
| GTA-PS$^{\alpha,\beta}$ | both SLEM-coupled (full) | Directed | Proactive | ✓ |

The ablation structure isolates three contributions independently: (i) topology signal $|\delta\mathbf{f}^{out}|, |\delta\mathbf{f}^{in}|$ vs. no topology (GTA-PS base vs. Dyna-PS); (ii) mixing schedule $\alpha_t$ vs. fixed $\alpha$ (GTA-PS$^{\alpha}$ vs. GTA-PS base); (iii) topology weight schedule $\beta_t$ vs. fixed $\beta$ (GTA-PS$^{\beta}$ vs. GTA-PS base). GTA-PS$^{\alpha,\beta}$ tests whether the two schedules interact constructively.

**Metrics.**

- **Value error** $\|V_t^K - V_t^*\|_\infty$: sup-norm distance from the optimal value function for the current reward $r_t$, measured after $K$ backups post-shift. Primary metric for RQ1 and RQ3; reported as a curve in $K$.
- **Replanning steps to $\varepsilon$-accuracy** $K_\varepsilon$: minimum $K$ such that $\|V_t^K - V_t^*\|_\infty \le \varepsilon\|V_t^*\|_\infty$. Scalar summary of backup efficiency (lower is better); default $\varepsilon = 0.1$.
- **Cumulative discounted return** $\sum_{t=0}^{T}\gamma^t r_t(s_t)$ under the greedy policy derived from $V_t$ after each $K$-step replanning period. Measures end-to-end policy quality and downstream impact.
- **Schedule-allocation diagnostic** (RQ2, RQ3): For each configuration, report $(\overline{\mathrm{SLEM}}_k,\ \bar{\alpha}_k,\ \bar{\beta}_k)$, the average number of states whose score increases under backward mixing ($\tau_t^{in}(s) > \tau_t^{out}(s)$), and the backup-count advantage $\Delta K_{\varepsilon,k}$ over both Dyna-PS and fixed-schedule GTA-PS. This tests whether the adaptive schedules move in the direction predicted by geometry.
- **Geometry / asymmetry diagnostic** (RQ4, RQ5): measured conductance proxies, doorway-crossing MFPT, SCC count or condensation depth where meaningful, $\|u_2 - v_2\|_2$, and $\|G_{\varepsilon,t}^{in} - G_{\varepsilon,t}^{out}\|_\infty$. These quantify soft bottlenecks versus hard graph decomposition and near-reversible versus directed structure.
- **Curl–divergence diagnostic** (RQ6): operator-split norms $\|A_t\|_F$ and $\|I - S_t\|_F$, their ratio, and their correlation with $\Delta K_{\varepsilon,k}$ and top-$q$ bottleneck coverage. This tests whether performance is explained by circulation alone, dissipative structure alone, or their interaction.
- **SLEM tracking error** (RQ2): $|\widehat{\mathrm{SLEM}}_t - \mathrm{SLEM}_t|$ comparing first-order incremental tracking to full Arnoldi recomputation. Confirms that drift is small enough for the adaptive schedules to remain reliable.

**Implementation Details.**

- *Host planner*: synchronous full-backup DP (known $p$) for MiniGrid-FourRooms and GARNET; Dyna-style model-based planning with learned or discretized local dynamics for Safety-Gymnasium CarGoal and MuJoCo Reacher.
- *Resolvent parameter*: $\varepsilon = \varepsilon^* = (1-\gamma)/\gamma$ (canonical alignment of $\mathbf{f}^{out}$ with the value function, Definition 6). Ablated over $\varepsilon \in \{0.01, \varepsilon^*, 1.0\}$ on GARNET.
- *Backup budget*: $K \in \{\frac{1}{10}|\mathcal{S}|, \frac{1}{2}|\mathcal{S}|, |\mathcal{S}|\}$ per shift epoch; $K = \frac{1}{10}|\mathcal{S}|$ is the tight-budget regime.
- *GTA-PS hyperparameters*: temperature $\alpha$, $\beta$, $\beta_1$, $\beta_2$ grid-searched per environment and reward configuration.
- *Computational setup*: exact LU in the smallest tabular settings; restarted GMRES ($m = 20$, ILU(0)) or truncated Neumann approximation in larger sparse settings. The memo does not hard-code solver thresholds because throughput depends on memory layout, batching, and available parallel hardware.
- *SLEM recomputation*: Arnoldi ($j = 15$) every $T_{\mathrm{arn}} = \lceil|\mathcal{S}|^{1/2}\rceil$ steps with first-order tracking between.
- *Random seeds*: 1000 independent seeds; results as mean ± 95% CI. GARNET additionally averaged over 100 random MDP draws per $|\mathcal{S}|$.

- *Software*: NumPy/SciPy/PyTorch for linear algebra; Gymnasium (Towers et al., 2024); Safety-Gymnasium (Ji et al., 2023); MiniGrid (Chevalier-Boisvert et al., 2023); MuJoCo (Todorov et al., 2012); GARNET per Archibald et al. (1995).
- GPU acceleration is possible for the heavy linear algebra, but the memo focuses on algorithmic and instruction-level complexity rather than wall-clock runtime, which depends on implementation choices.

**Expected Outcomes and Failure Modes.**

- *RQ1 (proactiveness)*: GTA-PS should outperform Dyna-PS at small and moderate backup budgets because the topology signal is available immediately after each reward shift, whereas reactive residuals remain MFPT-limited. The advantage should be largest when $K < t_{\text{mix}}(1/4)$ or, equivalently as a rough experimental predictor, when $T_{\text{shift}}/t_{\text{mix}}(1/4)$ is small.
- *RQ2 (allocation)*: the fully adaptive GTA-PS$^{\alpha,\beta}$ schedule should outperform or match the best fixed allocation once geometry varies across shifts; in stable, low-SLEM regimes the gain over a well-tuned fixed schedule should shrink. The first $K$ queue pops are also expected to remain more correlated with the one-shot topology ranking when shifts are fast or $K$ is small.
- *RQ3 (convergence / propagation tradeoff)*: forward-heavy rankings should maximize top-1 agreement with the decision oracle and converge faster intra-shift, while larger backward weight may increase coverage and faster recovery inter-shift. The best finite-$K$ performance should come from an intermediate accuracy.
- *RQ4 (symmetric vs. asymmetric effects)*: the gain of directed GTA-PS over symmetric-Laplacian and forward-only controls should grow with measured non-reversibility and shrink when $u_2 \approx v_2$ and $G^{in}_{\varepsilon,t} \approx G^{out}_{\varepsilon,t}$.
- *RQ5 (bottleneck / SCC geometry)*: the largest advantage should appear in geometrically bottlenecked environments, including single-SCC environments where TVI offers no decomposition benefit but reactive PS is still slowed by doorway or MFPT barriers.
- *RQ6 (curl / divergence interaction)*: the strongest directional gains should occur when the antisymmetric component $A_t$ is substantial while the dissipative part $I - S_t$ still creates separable source–sink geometry. Curl without bottlenecked divergence should mostly create cycling, while nearly symmetric diffusion should collapse toward the undirected baseline.
- *Failure mode (coarse approximate resolvent, large $|\mathcal{S}|$)*: If the iterative or truncated Neumann approximation is too crude, the topology field may blur the top-$q$ ordering and weaken the proactive advantage. The large GARNET settings will quantify how much approximation error can be tolerated before ranking fidelity degrades materially.

**Checklist**

- **MDP axes to check:** periodic / aperiodic; symmetric / asymmetric (or reversible / non-reversible); cycle-like / DAG-like dominant flow; sparse / dense transition support; deterministic / stochastic transition kernel.
- **MDP axes to ablate:** at least one experiment family should deliberately sweep each of the above axes instead of treating them as fixed environment metadata.
- **Agent ablations to check:** residual choice (Bellman residual, TD-style residual, GTA-augmented residual); planning budget $K$; scheduler choice (fixed $(\alpha, \beta)$, forward-only $\alpha_t$=0, adaptive $\alpha_t$, adaptive $\beta_t$, GTA-PS$^{\alpha,\beta}$).
- **Budget reporting to check:** report $K$ both in absolute form and relative to the propagation timescale through $K/t_{\text{mix}}(1/4)$ whenever this quantity can be estimated.
- **Nonstationary reward process to check:** reward support, reward magnitude law, shift frequency $T_{\text{shift}}$, local vs. global shifts, and random vs. scheduled vs. adversarial shift mechanism.
- **Shift-regime ablations to check:** include at least one fast-shift regime and one slow-shift regime so the heuristic predictor $T_{\text{shift}}/t_{\text{mix}}(1/4)$ can be tested explicitly.
- **Environment coverage to check:** at least one maze / navigation domain; at least one robotics / logistics / scheduling / routing-style task; at least one synthetic generator such as GARNET; every environment must include a clear nonstationary reward mechanism.
- **Statistical protocol to check:** number of independent trials / seeds, confidence-interval construction, aggregation rule (mean and, if needed, median), and whether reward-shift realizations are shared across methods.

- **Fair-comparison controls to check:** same host planner, same transition-model access, same reward-shift schedule, and same planning budget across all compared methods.
- **Performance measurements to check:** post-shift value error, $K_\varepsilon$, cumulative return, and runtime / memory cost of the chosen resolvent realization.
- **Mechanism measurements to check:** SLEM and SLEM-tracking error; conductance / MFPT proxies; $\|u_2 - v_2\|_2$; $\|G^{in}_{\varepsilon,t} - G^{out}_{\varepsilon,t}\|_\infty$; overlap or rank-correlation between the one-shot topology ranking and the first $K$ queue queries/pops, especially when $K$ is small or shifts are fast.
- **Approximation-control checks:** if GMRES, truncated Neumann, or any other approximate resolvent is used, report tolerance / truncation depth and compare ranking fidelity against an exact or high-accuracy reference on a smaller matched instance.
- **Negative controls to check:** near-reversible or nearly symmetric environments, stationary-reward controls, and large-budget regimes where reactive PS has enough time to propagate.

**Results**

**Conclusion & Further works**

**References**


T. W. Archibald, K. I. M. McKinnon, and L. C. Thomas. On the generation of Markov decision processes. *Journal of the Operational Research Society*, 46(3):354–361, 1995. doi: 10.1057/jors.1995.55. GARNET: randomised tabular MDPs with controlled $|S|$, $|A|$, branching factor; standard PS benchmark.

Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo Perez-Vicente, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Doina Precup. Minigrid & Miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL `https://arxiv.org/abs/2306.13831`. Provides `MiniGrid-FourRooms-v1` and flexible goal-relocation wrappers.

Fan Chung and Shing-Tung Yau. Discrete Green's functions. *Journal of Combinatorial Theory, Series A*, 91 (1-2):191–214, 2000. doi: 10.1006/jcta.2000.3094. Foundational properties of Green's functions on finite graphs, including their relation to the graph Laplacian, spectral components, and the fundamental matrices of a graph.

Peng Dai, Mausam, Daniel S. Weld, and Judy Goldsmith. Topological value iteration algorithms. *Journal of Artificial Intelligence Research*, 42:181–209, 2011. doi: 10.1613/jair.3390. URL `https://arxiv.org/abs/1401.3910`. Introduces TVI (SCC-based backup ordering) and FTVI (focused TVI with heuristic pruning).

Jiaming Ji, Borong Zhang, Jiayi Zhou, Xuehai Pan, Weidong Huang, Ruiyang Sun, Yiran Geng, Yifan Zhong, Juntao Dai, and Yaodong Yang. Safety-gymnasium: A unified safe reinforcement learning benchmark. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL `https://arxiv.org/abs/2310.12567`. Introduces the Safety-Gymnasium benchmark family, including car-based goal-reaching tasks with hazards such as `SafetyCarGoal`.

Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. doi: 10.1016/S0004-3702(99)00052-1. Introduces the Four-Rooms gridworld benchmark.

Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033. IEEE, 2012. doi: 10.1109/IROS.2012.6386109. Canonical reference for MuJoCo control tasks including the planar reaching benchmark `Reacher`.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Higueras, Andrea Pierrot, Sander Schulhoff, Jun Jet Tan, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL `https://arxiv.org/abs/2407.17032`. Provides `FrozenLake-v1` (stochastic, known $p$) and `Taxi-v3`.